CÓDIGOCLARO · RUTA SQL · ORACLE, MYSQL Y POSTGRESQL

SQL relacional: Oracle, MySQL y PostgreSQL

15 temas · ejemplos etiquetados por dialecto · documentación oficial enlazada

Las diferencias entre motores se señalan en cada tema y se enlazan a la documentación del producto correspondiente.

Tema 01 · Modelo relacional y tipos

Objetivo

Diseñar tablas pequeñas, elegir tipos que describan los datos y expresar las relaciones entre alumnos, cursos y matrículas.

Explicación

Una tabla representa filas con atributos definidos por columnas. En una relación muchos-a-muchos, una tercera tabla, aquí matrícula, conecta las entidades y puede guardar datos propios de esa relación, como la fecha de alta.

El tipo limita qué valores tienen sentido: un precio requiere un decimal exacto, un recuento suele ser entero y una fecha de calendario no es texto libre. La elección concreta del tipo depende del motor; no hay que asumir que DATE o los tipos de texto significan exactamente lo mismo en todos los dialectos.

Ejemplo guiado

Esquema didáctico · PostgreSQL

```
CREATE TABLE alumno (
  id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  nombre varchar(100) NOT NULL,
  email varchar(254) UNIQUE
);
CREATE TABLE curso (
  id integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  titulo text NOT NULL,
  activo boolean NOT NULL DEFAULT true,
  precio numeric(8,2) NOT NULL CHECK (precio >= 0)
);
CREATE TABLE matricula (
  alumno_id integer REFERENCES alumno(id),
  curso_id integer REFERENCES curso(id),
  fecha date NOT NULL, fecha_finalizacion date,
  PRIMARY KEY (alumno_id, curso_id)
);
```

Lectura

La clave compuesta impide duplicar la matrícula de la misma persona en el mismo curso. Las referencias impiden que matrícula apunte a un alumno o curso inexistente. email admite NULL: UNIQUE no equivale a NOT NULL.

Alcance y precisión

Correspondencias aproximadas, no equivalencias completas: PostgreSQL ofrece numeric/text/date; MySQL ofrece DECIMAL/VARCHAR/DATE; Oracle ofrece NUMBER/VARCHAR2/DATE. En Oracle, DATE incluye hora hasta segundos. Verifica precisión, límites y zona horaria en el motor elegido.

Práctica

Añade una tabla de docente y relaciona cada curso con un docente. Decide si un curso puede tener cero docentes, uno o varios y traduce esa regla a claves y restricciones.

Comprobación

Una matrícula es una entidad separada porque relaciona dos entidades y almacena fecha. Comprueba si tu diseño permite repetir una matrícula por curso y si esa posibilidad es intencionada.

Fuentes enlazadas: Tipos PostgreSQL · Tipos MySQL 8.4 · Tipos Oracle 26

Tema 02 · SELECT: columnas, expresiones y alias

Objetivo

Construir una salida deliberada: escoger columnas, calcular expresiones y dar nombres comprensibles al resultado.

Explicación

SELECT proyecta columnas y expresiones de las filas que aporta FROM. El alias cambia el nombre de una columna en el resultado; no renombra la columna almacenada. Seleccionar solo lo necesario hace explícito el contrato de salida de la consulta.

Para ejercicios rápidos, SELECT * es cómodo; en consultas que alimentan una aplicación, una lista explícita evita que añadir una columna a la tabla altere accidentalmente la forma del resultado.

Ejemplo guiado

Consulta de lectura · PostgreSQL

```
SELECT c.id,
       c.titulo AS curso,
       c.precio,
       c.precio * 1.10 AS precio_con_recargo
FROM curso AS c;
```

Lectura

c es un alias de tabla que permite escribir c.titulo sin repetir el nombre completo. precio_con_recargo es una expresión calculada para el resultado; el ejemplo usa un 10 % ficticio y no representa una regla fiscal.

Alcance y precisión

Las expresiones se evalúan con las reglas de tipos y conversiones del motor. Si una columna usa un tipo entero, decimal o aproximado, consulta la documentación del dialecto antes de depender de precisión o redondeo.

Práctica

Muestra id y título de cada curso, y calcula un indicador de si el precio supera 50. Usa un alias que se entienda leyendo solo el resultado.

Comprobación

La consulta devuelve una fila por fila fuente de curso; el alias cambia el encabezado del resultado, no actualiza la tabla.

Fuentes enlazadas: Tutorial SELECT PostgreSQL · SELECT MySQL 8.4

Tema 03 · WHERE, operadores y NULL

Objetivo

Filtrar filas con condiciones booleanas y tratar explícitamente los valores ausentes.

Explicación

WHERE conserva las filas cuya condición resulta verdadera. Las condiciones pueden combinarse con AND, OR y NOT; los paréntesis ayudan a expresar la intención cuando se mezclan. En PostgreSQL, BETWEEN incluye ambos extremos del intervalo.

NULL significa que falta o no se conoce un valor. Compararlo con = NULL no produce verdadero: la comparación resulta desconocida. Usa IS NULL o IS NOT NULL. Por eso una fila con NULL no pasa un filtro como precio = 0 ni tampoco pasa su negación precio <> 0.

Ejemplo guiado

Dos filtros separados · PostgreSQL

```
SELECT id, titulo FROM curso
WHERE activo = true AND precio BETWEEN 20 AND 60;

SELECT alumno_id, curso_id FROM matricula
WHERE fecha_finalizacion IS NULL;
```

Lectura

La primera condición usa el estado y el intervalo del precio. La segunda encuentra matrículas sin fecha de finalización. No escribas fecha_finalizacion = NULL: IS NULL es la prueba definida para ese caso.

Alcance y precisión

Los ejemplos usan boolean de PostgreSQL. MySQL y Oracle tienen diferencias de tipos y literales; además, Oracle trata actualmente una cadena de longitud cero como NULL y advierte que no se dependa de ello como equivalencia futura.

Práctica

Busca cursos activos cuyo precio sea menor de 30 o mayor de 100. Después añade una condición que descarte los títulos NULL, si el esquema permitiera esos valores.

Comprobación

Antes de evaluar el filtro, confirma si cada columna puede ser NULL. Recuerda que UNKNOWN en WHERE tampoco incluye una fila.

Fuentes enlazadas: Filtros y SELECT PostgreSQL · Comparaciones y NULL PostgreSQL · NULL en Oracle

Tema 04 · Ordenación y paginación

Objetivo

Ordenar resultados de forma legible y paginar sin depender de un orden accidental.

Explicación

SQL no promete un orden si no se especifica ORDER BY. Si dos filas empatan en el primer criterio, el orden entre ellas sigue sin quedar fijado; añade una clave única como último criterio cuando necesites una página reproducible.

LIMIT/OFFSET y FETCH/OFFSET limitan filas, pero no hacen que la consulta sea más barata por sí solos. En PostgreSQL, saltar un OFFSET grande sigue requiriendo computar las filas saltadas; para conjuntos grandes se suele diseñar paginación por clave.

Ejemplo guiado

Primeros 10 cursos desde el desplazamiento 20 · PostgreSQL

```
SELECT id, titulo
FROM curso
ORDER BY titulo ASC, id ASC
LIMIT 10 OFFSET 20;
```

Lectura

titulo fija el orden principal; id resuelve empates. Cambia 10 y 20 para probar otras páginas. Sin ORDER BY, incluso la primera página puede variar entre planes o ejecuciones.

Alcance y precisión

PostgreSQL y MySQL aceptan LIMIT 10 OFFSET 20. Oracle usa OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY. En los tres casos, usa ORDER BY con un criterio que resuelva empates si el orden debe ser determinista.

Práctica

Devuelve la página siguiente de cursos ordenada por precio y, en caso de empate, id. Escribe qué ocurre si eliminas el criterio id.

Comprobación

Hay 10 como máximo en la página; el desplazamiento empieza después de saltar 20 filas ordenadas. No supongas que la base devuelve las filas en orden de inserción.

Fuentes enlazadas: LIMIT/OFFSET PostgreSQL · SELECT y LIMIT MySQL 8.4 · FETCH/OFFSET Oracle 26

Tema 05 · Funciones de agregación

Objetivo

Resumir filas con COUNT, SUM, AVG, MIN y MAX, distinguiendo filas de valores no nulos.

Explicación

Una agregación transforma varias filas en un valor de resumen. COUNT(*) cuenta filas del conjunto; COUNT(columna) cuenta las filas cuyo valor en esa columna no es NULL. Esa diferencia es clave cuando el campo admite valores ausentes.

AVG, SUM, MIN y MAX aplican sus reglas de tipo y NULL por función y motor. En PostgreSQL, los agregados comunes ignoran entradas NULL; salvo COUNT, devuelven NULL cuando no hay filas. En PostgreSQL y MySQL, COUNT(*) cuenta filas y COUNT(columna) valores no NULL. No sustituyas automáticamente NULL por cero: decide si cero significa lo mismo en el dominio.

Ejemplo guiado

Total de matrículas y finalizaciones conocidas · PostgreSQL

```
SELECT COUNT(*) AS matriculas,
       COUNT(fecha_finalizacion) AS finalizadas
FROM matricula;
```

Lectura

matriculas cuenta todas las filas. finalizadas cuenta solo las que tienen fecha_finalizacion. En este esquema, una fecha nula representa una matrícula aún no finalizada; si la regla del negocio fuese otra, el cálculo tendría que cambiar.

Alcance y precisión

El tipo de retorno puede variar según función y tipos de entrada. Revisa la documentación del producto antes de asumir precisión, tipo de resultado o comportamiento de una agregación vacía.

Práctica

Calcula el precio medio y el precio máximo de los cursos. Decide qué debería mostrarse si todavía no hay cursos.

Comprobación

Con una tabla vacía, COUNT(*) produce 0 en los motores documentados; AVG no es un cero implícito. Comprueba la salida real en tu dialecto.

Fuentes enlazadas: Agregados PostgreSQL · Agregados MySQL 8.4 · COUNT Oracle 26

Tema 06 · GROUP BY y HAVING

Objetivo

Agrupar filas para obtener resúmenes por categoría y filtrar los grupos después de agregarlos.

Explicación

GROUP BY forma grupos que comparten los valores de las columnas indicadas. Una consulta de resumen puede incluir las columnas agrupadas y expresiones de agregación. HAVING filtra el resultado agrupado; WHERE filtra filas antes de que se formen los grupos.

Si necesitas incluir cursos que todavía no tienen matrículas, agrupar solo matricula no basta: el conjunto de entrada debe partir de curso y usar LEFT JOIN. Es importante distinguir ausencia de filas de una agregación cuyo resultado es cero.

Ejemplo guiado

Cursos con al menos diez matrículas · PostgreSQL

```
SELECT curso_id, COUNT(*) AS total
FROM matricula
GROUP BY curso_id
HAVING COUNT(*) >= 10
ORDER BY total DESC, curso_id;
```

Lectura

Cada fila de salida representa un curso_id presente en matrícula. HAVING COUNT(*) >= 10 conserva grupos que alcanzan el umbral; ORDER BY solo organiza la salida y no cambia qué grupos existen.

Alcance y precisión

Al portar consultas, evita depender de seleccionar columnas que no estén agrupadas ni agregadas: las reglas y extensiones pueden variar por motor y configuración. La forma mostrada sigue la regla explícita y portable.

Práctica

Obtén el total de matrículas por curso, pero solo para matrículas cuya fecha sea de 2026. ¿La condición de fecha va en WHERE o HAVING?

Comprobación

La condición sobre cada fila va en WHERE antes de agrupar; HAVING se reserva aquí para condiciones como COUNT(*) >= 10.

Fuentes enlazadas: SELECT, GROUP BY y HAVING PostgreSQL · GROUP BY y HAVING Oracle 26 · Agregados MySQL 8.4

Tema 07 · JOIN: combinar tablas relacionadas

Objetivo

Relacionar filas mediante claves y elegir entre coincidencias obligatorias y conservación de filas sin pareja.

Explicación

JOIN combina filas según una condición. INNER JOIN conserva las parejas que cumplen ON. LEFT JOIN también conserva cada fila de la tabla izquierda que no encontró pareja; en las columnas del lado derecho aparecen NULL.

La condición de emparejamiento se escribe en ON. Con un outer join, mover una condición del lado derecho a WHERE puede descartar las filas extendidas con NULL y cambiar el resultado práctico. Lee el ejemplo con cursos sin matrículas para notar la diferencia.

Ejemplo guiado

Mostrar también cursos sin matrícula · PostgreSQL

```
SELECT c.id, c.titulo, m.alumno_id
FROM curso AS c
LEFT JOIN matricula AS m
  ON m.curso_id = c.id
ORDER BY c.id, m.alumno_id;
```

Lectura

Un curso con tres matrículas aparece en tres filas. Un curso sin ninguna aparece una vez con alumno_id NULL. Si solo quieres cursos con matrículas, cambia a JOIN o INNER JOIN.

Alcance y precisión

Evita NATURAL JOIN en material didáctico introductorio: empareja automáticamente todas las columnas con igual nombre y el resultado puede cambiar al modificar el esquema. ON deja la relación visible.

Práctica

Añade el nombre del alumno y limita la salida a cursos con un título concreto. Prueba mover el filtro del curso a WHERE y luego a ON; observa por separado el efecto sobre los cursos sin pareja.

Comprobación

Antes de ejecutar, cuenta qué filas deben sobrevivir sin pareja. Esa decisión determina si necesitas INNER o LEFT y dónde se coloca cada predicado.

Fuentes enlazadas: Expresiones de tablas y JOIN PostgreSQL · Sintaxis SELECT y JOIN MySQL 8.4

Tema 08 · Subconsultas y EXISTS

Objetivo

Plantear preguntas de pertenencia y ausencia sin duplicar las filas externas.

Explicación

Una subconsulta puede expresar una condición dentro de otra consulta. EXISTS resulta verdadero cuando su subconsulta devuelve al menos una fila. Una subconsulta correlacionada puede referirse a la fila exterior mediante su alias.

Para saber si un alumno tiene alguna matrícula, EXISTS describe directamente la pregunta y mantiene como máximo una fila por alumno, aunque haya muchas coincidencias internas. NOT EXISTS expresa la ausencia. La base puede transformar internamente la consulta; el código describe el resultado, no obliga a un plan concreto.

Ejemplo guiado

Alumnos con al menos una matrícula · PostgreSQL

```
SELECT a.id, a.nombre
FROM alumno AS a
WHERE EXISTS (
  SELECT 1 FROM matricula AS m
  WHERE m.alumno_id = a.id
);
```

Lectura

La condición interna comprueba cada alumno. SELECT 1 comunica que solo interesa la existencia de una fila; el valor seleccionado no es el dato que se devuelve al usuario.

Alcance y precisión

NOT IN y NOT EXISTS no siempre son intercambiables si la subconsulta puede contener NULL: SQL usa lógica de tres valores. Para preguntas de existencia, EXISTS/NOT EXISTS evita esa ambigüedad semántica.

Práctica

Escribe una consulta de alumnos sin matrículas usando NOT EXISTS. Después crea un caso con dos matrículas para un alumno: ¿cuántas veces aparece con EXISTS?

Comprobación

El alumno aparece una sola vez por cada fila exterior que satisface la existencia; EXISTS no multiplica esa fila por cada matrícula encontrada.

Fuentes enlazadas: Subquery Expressions PostgreSQL · EXISTS Oracle 26

Tema 09 · CTE y consultas recursivas

Objetivo

Dividir una consulta compleja en pasos con WITH y entender el caso iterativo de una consulta recursiva.

Explicación

Una expresión de tabla común (CTE) da un nombre a una subconsulta disponible durante una sentencia. Ayuda a separar etapas y a explicar una consulta; no es, por el mero hecho de usar WITH, una promesa de que se guarde físicamente ni de que la consulta sea más rápida.

Una CTE recursiva combina un caso inicial (ancla) con un paso que vuelve a consultar el resultado anterior. El paso debe tener una condición de terminación. Es útil para jerarquías; sin un límite o condición correcta puede producir demasiadas filas o no terminar según el caso.

Ejemplo guiado

Generar la secuencia 1 a 5 · PostgreSQL

```
WITH RECURSIVE numeros(n) AS (
  SELECT 1
  UNION ALL
  SELECT n + 1 FROM numeros WHERE n < 5
)
SELECT n FROM numeros ORDER BY n;
```

Lectura

El ancla produce 1. Cada paso emite el número siguiente mientras n < 5; al llegar a 5 deja de producir filas. ORDER BY fija el orden que se muestra, no el orden intrínseco de evaluación.

Alcance y precisión

PostgreSQL y MySQL escriben WITH RECURSIVE para este patrón. Oracle usa factorización recursiva con WITH y una cláusula recursiva, sin esa palabra clave. La sintaxis de ciclo y búsqueda también difiere.

Práctica

Adapta el ejemplo para generar de 1 a 10 y declara en una frase cuál es el ancla, cuál el paso y cuál la condición que detiene la expansión.

Comprobación

La secuencia esperada contiene cinco filas. Si quitas WHERE n < 5, retira la ejecución y revisa el límite de recursión de tu motor antes de experimentar.

Fuentes enlazadas: CTE PostgreSQL · CTE MySQL 8.4 · Subconsulta recursiva Oracle 26

Tema 10 · INSERT, UPDATE y DELETE

Objetivo

Modificar datos indicando las columnas y filas afectadas, y verificar el alcance antes de confirmar cambios.

Explicación

INSERT crea filas, UPDATE modifica las que satisface WHERE y DELETE elimina las que satisface WHERE. En UPDATE y DELETE, omitir WHERE afecta a todas las filas; la sentencia es legal y puede ser intencionada, por lo que hay que revisar el filtro antes de ejecutarla.

Usa listas explícitas de columnas en INSERT para no depender del orden del esquema. El ejemplo muestra RETURNING de PostgreSQL para observar filas afectadas en la misma operación; esa cláusula no tiene sintaxis idéntica en todos los motores.

Ejemplo guiado

Plantillas independientes · PostgreSQL

```
INSERT INTO curso (titulo, precio)
VALUES ('SQL inicial', 25.00) RETURNING id;

UPDATE curso SET precio = 19.90
WHERE id = 42 RETURNING id, precio;

DELETE FROM curso WHERE id = 42
RETURNING id;
```

Lectura

Son alternativas de ejemplo, no un único guion: si ejecutas las tres en secuencia, el DELETE elimina el curso actualizado. Antes de un cambio real, prueba primero el predicado con SELECT y confirma que identifica las filas esperadas.

Alcance y precisión

RETURNING está documentado como extensión de PostgreSQL para estas sentencias. Los mecanismos para obtener el identificador generado y las formas de upsert cambian entre PostgreSQL, MySQL y Oracle.

Práctica

Escribe un UPDATE que desactive un curso solo cuando su id sea el elegido y esté activo. Antes, convierte la condición WHERE en un SELECT para estimar las filas afectadas.

Comprobación

Una respuesta segura contiene un filtro por id; si la vista previa devuelve más de una fila, no ejecutes el cambio hasta entender el motivo.

Fuentes enlazadas: INSERT PostgreSQL · UPDATE PostgreSQL · DELETE PostgreSQL

Tema 11 · Transacciones y ACID

Objetivo

Agrupar cambios relacionados y elegir entre COMMIT y ROLLBACK según el resultado comprobado.

Explicación

Una transacción permite tratar una unidad de cambios de forma controlada: COMMIT la confirma; ROLLBACK revierte el trabajo no confirmado. ACID nombra atomicidad, consistencia, aislamiento y durabilidad, pero las capacidades dependen del motor de almacenamiento y de la configuración.

En MySQL, InnoDB soporta transacciones ACID; no generalices automáticamente esa garantía a cualquier motor de almacenamiento. Oracle emite COMMIT implícitos alrededor de sentencias DDL; por ello, no traslades literalmente secuencias de DDL transaccionales entre motores.

Ejemplo guiado

Revisar un cambio antes de confirmarlo · PostgreSQL

```
BEGIN;
UPDATE curso SET precio = 19.90 WHERE id = 42;
SELECT id, precio FROM curso WHERE id = 42;
-- Si el resultado es correcto: COMMIT;
-- Si no lo es: ROLLBACK;
```

Lectura

Las líneas de COMMIT y ROLLBACK son decisiones alternativas. El ejemplo mantiene la transacción abierta mientras verificas el resultado; una aplicación debe cerrar correctamente cada transacción incluso ante errores.

Alcance y precisión

En PostgreSQL, BEGIN inicia un bloque de transacción. MySQL 8.4 documenta ACID para InnoDB. La documentación de Oracle describe COMMIT y ROLLBACK y advierte de los commits implícitos para DDL.

Práctica

Añade dos cambios que deban tener éxito juntos. Indica qué comprobación realizarías antes de COMMIT y qué caso debería provocar ROLLBACK.

Comprobación

Si uno de los cambios deja el conjunto de datos en un estado inválido, no confirmes solo la mitad: revierte la unidad y corrige la causa.

Fuentes enlazadas: Transacciones PostgreSQL · InnoDB y ACID MySQL 8.4 · Transacciones Oracle 26 · COMMIT Oracle 26

Tema 12 · Claves y restricciones

Objetivo

Declarar reglas de integridad en el esquema para que los datos válidos no dependan solo del código de aplicación.

Explicación

PRIMARY KEY identifica de forma única cada fila y no admite NULL en sus columnas. FOREIGN KEY exige que la referencia apunte a una clave válida de la tabla relacionada. NOT NULL requiere un valor; UNIQUE evita duplicados según las reglas del motor; CHECK valida una condición sobre los valores sometidos a la restricción.

Las restricciones viven junto a los datos y se comprueban al insertar o cambiar filas. En PostgreSQL, CHECK no debe usarse para validar otras filas o tablas: la documentación explica que esa suposición no garantiza consistencia ante cambios posteriores.

Ejemplo guiado

Reglas de integridad · PostgreSQL

```
CREATE TABLE curso (
  id integer PRIMARY KEY,
  titulo text NOT NULL,
  precio numeric(8,2) CHECK (precio >= 0),
  codigo text UNIQUE
);
CREATE TABLE matricula (
  alumno_id integer REFERENCES alumno(id),
  curso_id integer REFERENCES curso(id),
  PRIMARY KEY (alumno_id, curso_id)
);
```

Lectura

La primera tabla prohíbe precios negativos y códigos duplicados. La segunda usa clave compuesta para evitar dos matrículas de la misma pareja. Si una referencia debe ser obligatoria, añade NOT NULL a cada columna de la clave foránea.

Alcance y precisión

El soporte de restricciones puede depender del motor o almacenamiento. En MySQL 8.4, las claves foráneas se implementan por motores como InnoDB; no asumas el mismo comportamiento con cualquier storage engine.

Práctica

Añade una regla que limite el título a contenido obligatorio y otra que prohíba precio negativo. Describe qué error de datos bloquea cada una.

Comprobación

Las reglas de una fila caben en CHECK; la existencia de una fila relacionada corresponde a FOREIGN KEY; la identidad corresponde a PRIMARY KEY.

Fuentes enlazadas: Constraints PostgreSQL · Foreign keys MySQL 8.4 · Constraints Oracle 26

Tema 13 · Índices y EXPLAIN

Objetivo

Relacionar índices con patrones de consulta y leer el plan como una estimación que debe contrastarse.

Explicación

Un índice es una estructura auxiliar que puede ayudar a localizar filas o satisfacer partes de una consulta. No es una orden para el planificador: según el tamaño, la selectividad, las estadísticas y los costes, el motor puede preferir un escaneo secuencial.

Los índices ocupan espacio y hay que mantenerlos al insertar, actualizar o eliminar. Añadir uno por intuición puede encarecer escrituras sin beneficiar consultas reales. EXPLAIN permite inspeccionar el plan previsto; las variantes que ejecutan y miden también ejecutan la consulta, por lo que hay que conocer sus efectos antes de usarlas con DML.

Ejemplo guiado

Crear un índice de búsqueda y consultar el plan · PostgreSQL

```
CREATE INDEX idx_matricula_curso
  ON matricula (curso_id);

EXPLAIN
SELECT alumno_id FROM matricula
WHERE curso_id = 4;
```

Lectura

El índice puede servir a filtros y uniones por curso_id. La salida de EXPLAIN muestra nodos del plan y estimaciones; que no aparezca un Index Scan no prueba por sí solo un error.

Alcance y precisión

PostgreSQL documenta EXPLAIN y EXPLAIN ANALYZE por separado; las variantes ANALYZE ejecutan la sentencia. MySQL 8.4 tiene su propio alcance y salida. Oracle EXPLAIN PLAN guarda el plan estimado en una tabla y puede diferir del plan de ejecución real. No compares literalmente nombres de nodos entre motores.

Práctica

Busca una consulta frecuente del proyecto. Anota el filtro o unión, predice qué índice podría ayudar y luego inspecciona el plan con datos representativos.

Comprobación

Una mejora real compara el mismo caso antes y después, considera tiempo y escrituras, y confirma que el plan usa una estrategia razonable para ese volumen.

Fuentes enlazadas: Introducción a índices PostgreSQL · EXPLAIN PostgreSQL · EXPLAIN MySQL 8.4 · EXPLAIN PLAN Oracle 26

Tema 14 · Vistas y lógica en el servidor

Objetivo

Usar vistas para dar un nombre a consultas reutilizables y reconocer que rutinas y triggers son específicos del motor.

Explicación

Una vista normal define una consulta con un nombre reutilizable. En PostgreSQL, la vista no se materializa físicamente: la consulta se ejecuta cuando se referencia. Una vista materializada es otro objeto y guarda resultados que deben actualizarse según las reglas del motor.

Procedimientos, funciones y disparadores permiten lógica cercana a los datos, pero su lenguaje, delimitadores, tipos de retorno y reglas de ejecución no se portan automáticamente. Antes de diseñar esa lógica, define qué debe garantizar la base y qué pertenece a la aplicación; después consulta el manual exacto de la versión.

Ejemplo guiado

Vista reutilizable · PostgreSQL

```
CREATE VIEW detalle_matriculas AS
SELECT a.id AS alumno_id, a.nombre,
       c.id AS curso_id, c.titulo, m.fecha
FROM matricula AS m
JOIN alumno AS a ON a.id = m.alumno_id
JOIN curso AS c ON c.id = m.curso_id;

SELECT * FROM detalle_matriculas;
```

Lectura

La vista proporciona un nombre y una forma de salida a la unión; no copia por defecto esas filas a una tabla independiente en PostgreSQL.

Alcance y precisión

No presentes CREATE PROCEDURE ni la sintaxis de triggers como SQL universal. PostgreSQL, MySQL y Oracle documentan sintaxis y capacidades distintas para rutinas y disparadores.

Práctica

Crea una vista que muestre solo cursos activos. Explica quién puede consultarla y qué cambios en las tablas base afectarían su resultado.

Comprobación

Una vista normal vuelve a consultar las tablas subyacentes; si necesitas datos almacenados, investiga materialized views o el objeto equivalente de tu motor.

Fuentes enlazadas: CREATE VIEW PostgreSQL · CREATE VIEW MySQL 8.4 · CREATE VIEW Oracle 26 · Vista materializada PostgreSQL · Rutinas MySQL 8.4 · Procedimientos Oracle 26 · CREATE TRIGGER PostgreSQL

Tema 15 · Proyecto integrador y portabilidad

Objetivo

Resolver un caso completo y separar reglas relacionales comunes de sintaxis propia de cada producto.

Explicación

Diseña el catálogo y sus matrículas con claves y restricciones; escribe una consulta de lectura, otra de resumen y una modificación protegida por transacción. Documenta el resultado esperado con datos de ejemplo pequeños, incluyendo ausencia de matrículas, valores NULL y duplicados que deban rechazarse.

Al pasar la solución de PostgreSQL a MySQL u Oracle, revisa tipos, generación de claves, paginación, funciones, DDL, transacciones y extensiones. Una consulta parecida no implica que cada detalle del resultado, precisión, bloqueo o plan sea idéntico.

Ejemplo guiado

Comparación puntual de sintaxis

```
-- PostgreSQL / MySQL: pagina ordenada
SELECT id, titulo FROM curso
ORDER BY titulo, id LIMIT 10 OFFSET 20;

-- Oracle: la misma idea de paginación
SELECT id, titulo FROM curso
ORDER BY titulo, id
OFFSET 20 ROWS FETCH NEXT 10 ROWS ONLY;
```

Lectura

El código está separado en dos alternativas; ejecútalo solo en el motor indicado. El orden por id desempata títulos repetidos. Las cláusulas de paginación son parecidas entre PostgreSQL y MySQL en esta forma, pero Oracle usa FETCH/OFFSET.

Alcance y precisión

Generación automática de id: PostgreSQL y Oracle tienen columnas IDENTITY; MySQL usa AUTO_INCREMENT. Tipos, excepciones y opciones difieren: trata el esquema como una migración revisable, no como sustitución de palabras.

Práctica

Entrega un diagrama con tres tablas, el DDL de PostgreSQL, una consulta LEFT JOIN, un resumen con GROUP BY, una consulta NOT EXISTS y una modificación transaccional. Marca qué fragmentos cambiarían para MySQL y Oracle.

Comprobación

Revisión final: integridad referencial, filtro NULL correcto, orden total para paginar, condición WHERE comprobada antes de DML, y plan contrastado con datos representativos.

Fuentes enlazadas: LIMIT PostgreSQL · SELECT MySQL 8.4 · Row limiting Oracle 26 · Identity PostgreSQL · AUTO_INCREMENT MySQL 8.4 · Identity Oracle 26

Proyecto integrador · matrículas de módulos

Objetivo

Diseña una base pequeña para gestionar alumnos, módulos y matrículas. La práctica se ejecuta en una base local desechable; antes de cambiar un sistema real, revisa permisos, copias y dialecto.

Pasos de trabajo

1. Dibuja las tablas alumno, modulo y matricula; marca claves primarias, claves foráneas y la regla que impide duplicar la misma matrícula.

2. Escribe el DDL para el motor elegido y etiqueta cualquier diferencia de identity, tipos de texto o generación de claves.

3. Inserta dos alumnos y dos módulos. Añade matrículas dentro de una transacción y consulta el resultado con JOIN.

4. Prepara una consulta parametrizada para buscar matrículas por alumno; no unas valores de usuario mediante concatenación de cadenas.

Comprobaciones

- Un id de alumno inexistente se rechaza por la clave foránea.

- Una matrícula duplicada no crea una segunda relación.

- El informe muestra cero, una o varias matrículas sin cambiar el esquema.

Explicación final

Compara el SQL común con el dialecto concreto que ejecutaste. Anota qué sintaxis depende de PostgreSQL, MySQL u Oracle y comprueba las diferencias en la documentación oficial enlazada antes de reutilizarla.